

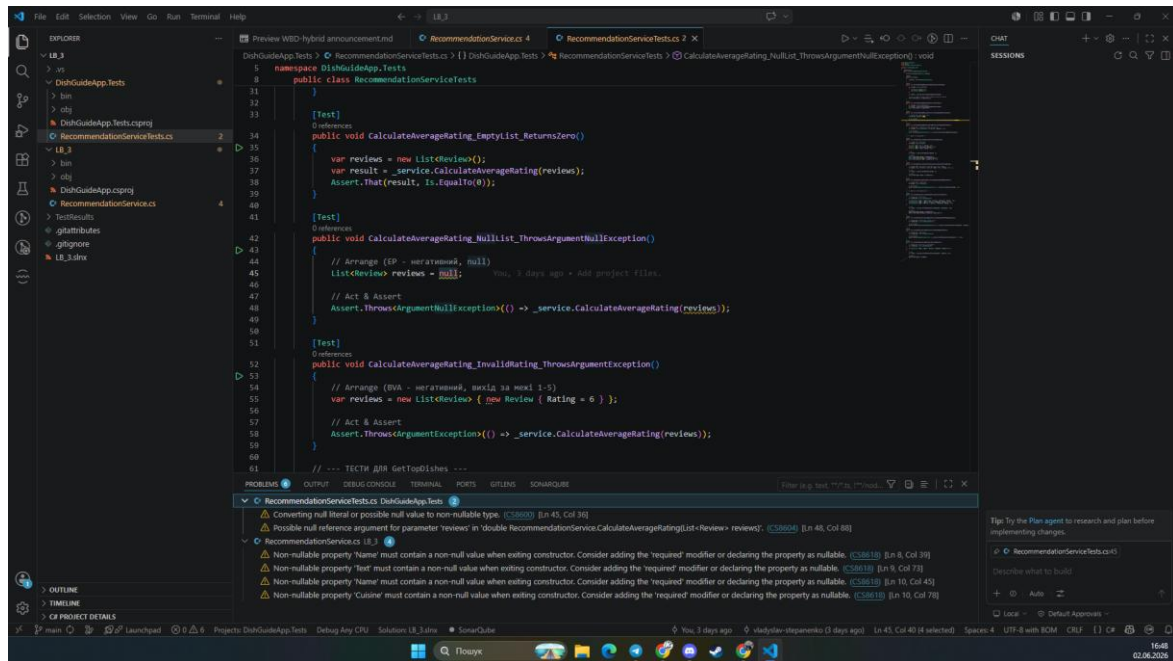
ЛАБОРАТОРНА РОБОТА 4

РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

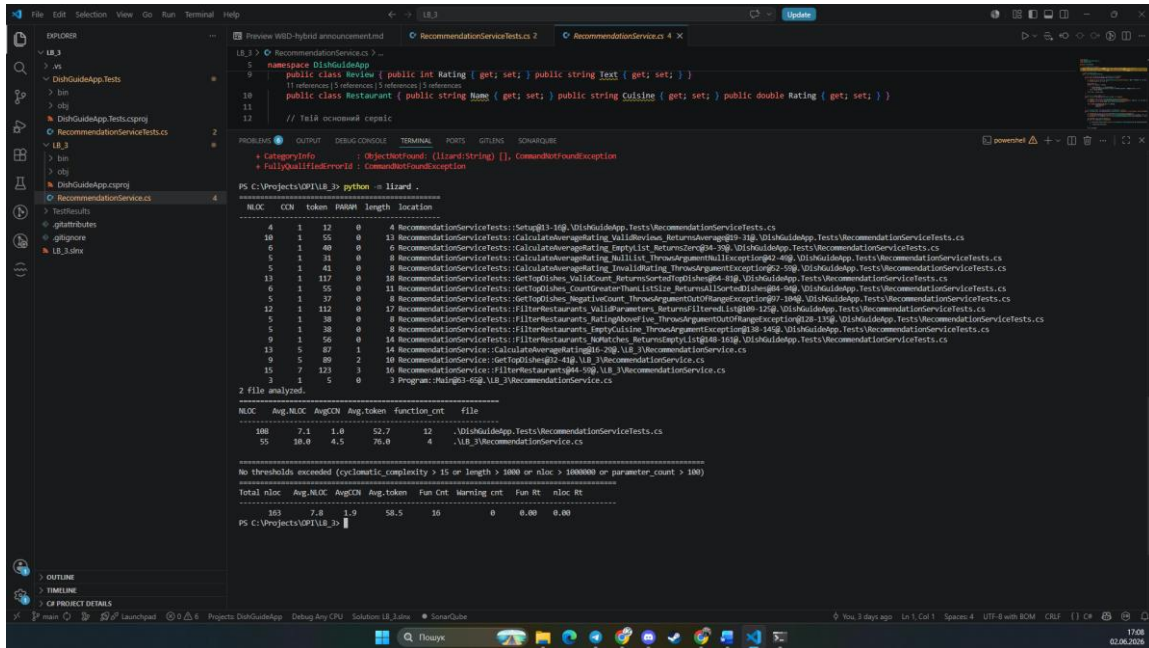
4.1 Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

2. Встановити SonarQube for IDE (SonarLint) у VS Code. Запустити аналіз свого коду та зафіксувати початкову кількість issues (скріншот «до»).



3. Запустити лінтер (Ruff для Python: `ruff check src/`; ESLint для JS/TS; Checkstyle для Java). Зафіксувати кількість попереджень.



6. Провести Code Review: ретельно проаналізувати код одногрупника та виявити щонайменше 5 проблем якості коду.

Результати Code Review одногрупника:

№	Рядок коду	Проблема	Категорія (Code smell)	Рекомендація
1	myTest.cs: 14, 21, 28...	Дублювання створення об'єкта Restaurant у кожному з 10 тестів.	Duplicated Code	Винести ініціалізацію базового об'єкта Restaurant у конструктор тестового класу (або метод Setup), щоб уникнути повторення коду.

№	Рядок коду	Проблема	Категорія (Code smell)	Рекомендація
2	Restaurant.cs: 14	Публічний доступ до мутації списку Dishes. Хоча set є приватним, сам об'єкт List можна непомітно змінити ззовні через res.Dishes.Add().	Encapsulation Violation	Змінити публічний тип властивості з List<string> на IReadOnlyList<string>, щоб гарантувати незмінність колекції ззовні.
3	Restaurant.cs: 40, 56	Використання Console.WriteLine всередині методів бізнес-логіки. Клас моделі даних не повинен напряду взаємодіяти з консоллю.	SRP Violation (Inappropriate Intimacy)	Видалити Console.WriteLine з методів. Клас має лише керувати даними, а логування чи вивід на екран — завдання зовнішнього коду.
4	Restaurant.cs: 11, 12	Властивості Name та Address оголошені як string, але можуть викликати попередження статичного аналізатора (CS8618) щодо null-безпеки.	Null Safety (Language Misuse)	Додати модифікатор required до цих властивостей або змінити їх тип на nullable string?.

№	Рядок коду	Проблема	Категорія (Code smell)	Рекомендація
5	Restaurant.cs: 65	Маскування помилки (Error Hiding): якщо передати від'ємну кількість страв, метод мовчки змінює її на 0 замість повідомлення про помилку.	Error Hiding	Замінити <code>if (count < 0) count = 0;</code> на викидання винятку <code>throw new ArgumentOutOfRangeException(nameof(count));</code> .

7. Кожну проблему задокументувати як inline-коментар у PR або GitHub Issue із зазначенням: (а) рядок коду; (б) категорія code smell; (в) конкретна рекомендація щодо виправлення.

Посилання на PR з коментарями Code Review (URL):

<https://github.com/oleksandrri/LaboratoryOPI3/pull/2#pullrequestreview-4410684492>

4. Результати рефакторингу власного коду: для кожної операції — «БУЛО (фрагмент коду) → СТАЛО (фрагмент коду) → ЧОМУ (обґрунтування)». Мінімум 3 операції.

Операція 1: Усунення попереджень аналізатора щодо null (Null Safety)

- **БУЛО (фрагмент коду):**

// У файлі RecommendationServiceTests.cs

```
List<Review> reviews = null;
```

```
Assert.Throws<ArgumentNullException>(() =>
    _service.CalculateAverageRating(reviews));
```

- **СТАЛО (фрагмент коду):**

```
// Змінну reviews видалено, використано null-forgiving operator
Assert.Throws<ArgumentNullException>(() =>
_service.CalculateAverageRating(null!));
```

- **ЧОМУ (обґрунтування):** Усунуто попередження статичного аналізатора CS8600 та CS8604. Використання оператора null-forgiving (!) є виправданим та безпечним у цьому контексті, оскільки передача null є очікуваною поведінкою негативного тесту для перевірки генерації винятку `ArgumentNullException`. Це зробило тест чистішим.

Операція 2: Видалення магічних чисел (Magic Numbers)

- **БУЛО (фрагмент коду):**

```
// У файлі RecommendationService.cs
if (review.Rating < 1 || review.Rating > 5)
    throw new ArgumentException("Рейтинг у відгуку має бути в діапазоні від 1 до 5");
```

- **СТАЛО (фрагмент коду):**

```
// Додано константи на рівні класу
private const int MinReviewRating = 1;
private const int MaxReviewRating = 5;

// В оновленому методі
if (review.Rating < MinReviewRating || review.Rating > MaxReviewRating)
    throw new ArgumentException($"Рейтинг у відгуку має бути в діапазоні від {MinReviewRating} до {MaxReviewRating}");
```

- **ЧОМУ (обґрунтування):** Усунуто антипатерн "магічні числа" (Magic Numbers) у методах валідації. Введення іменованих констант робить код більш самодокументованим і значно полегшує його підтримку. Якщо в майбутньому шкала оцінювання зміниться (наприклад, від 1 до 10), зміни потрібно буде внести лише в одному місці класу.

Операція 3: Заміна імперативного циклу на декларативний LINQ-запит

- **БУЛО (фрагмент коду):**

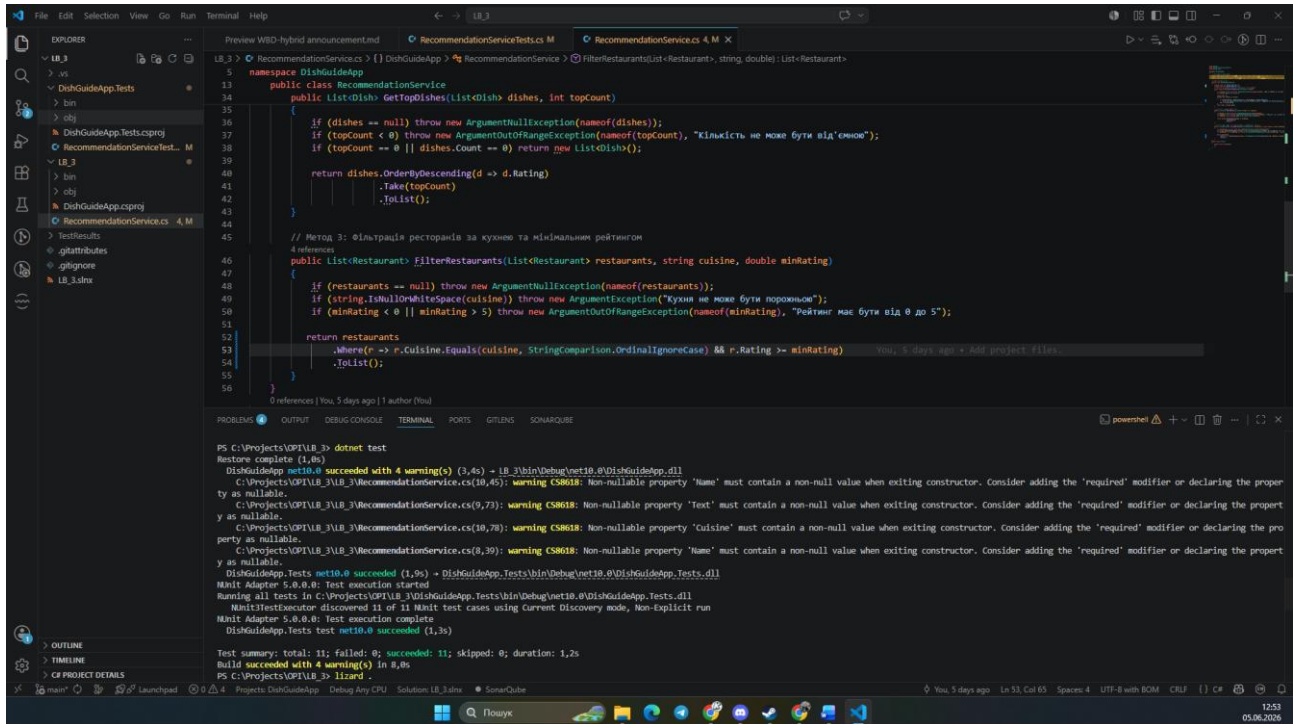
```
// У методі FilterRestaurants  
  
var filtered = new List<Restaurant>();  
  
foreach (var restaurant in restaurants)  
{  
    if (restaurant.Cuisine.Equals(cuisine, StringComparison.OrdinalIgnoreCase) &&  
        restaurant.Rating >= minRating)  
    {  
        filtered.Add(restaurant);  
    }  
}  
  
return filtered;
```

- **СТАЛО (фрагмент коду):**

```
return restaurants  
    .Where(r => r.Cuisine.Equals(cuisine, StringComparison.OrdinalIgnoreCase) &&  
        r.Rating >= minRating)  
    .ToList();
```

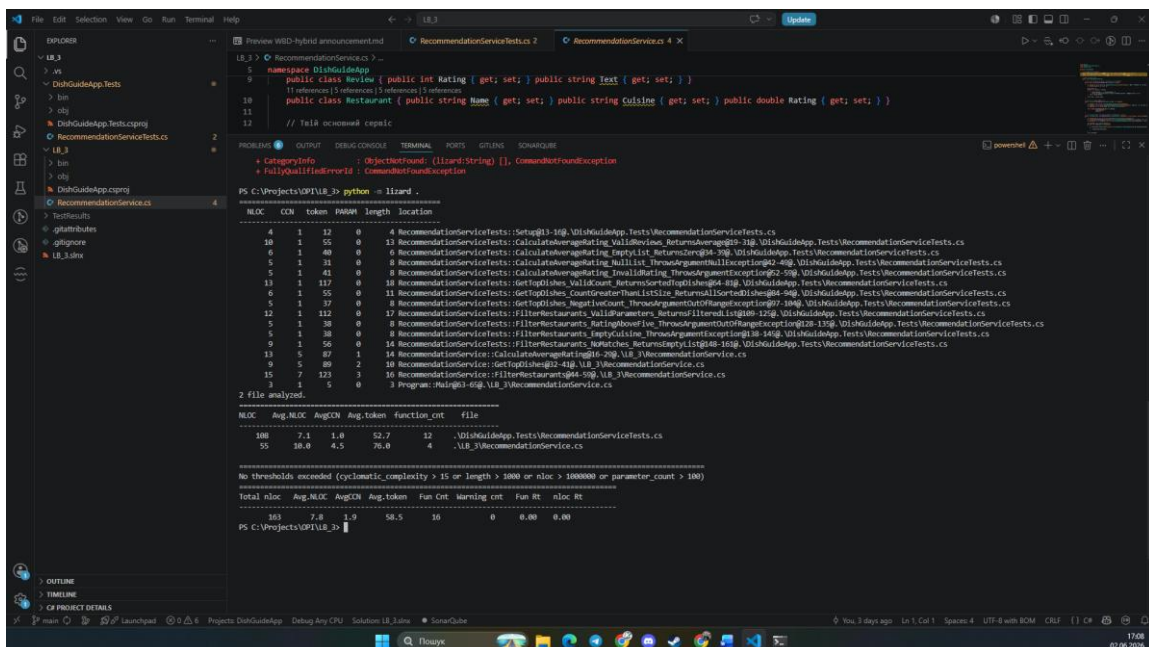
- **ЧОМУ (обґрунтування):** Багатослівний цикл `foreach` та внутрішня умовна конструкція `if` були замінені на декларативний підхід за допомогою методу розширення LINQ `Where`. Це зменшило цикломатичну складність методу, позбавило необхідності створювати проміжний список `filtered` і зробило код лаконічнішим та значно легшим для читання.

5. Звіт регресійного тестування: скріншот результату pytest / JUnit / Jest — усі тести green після рефакторингу.

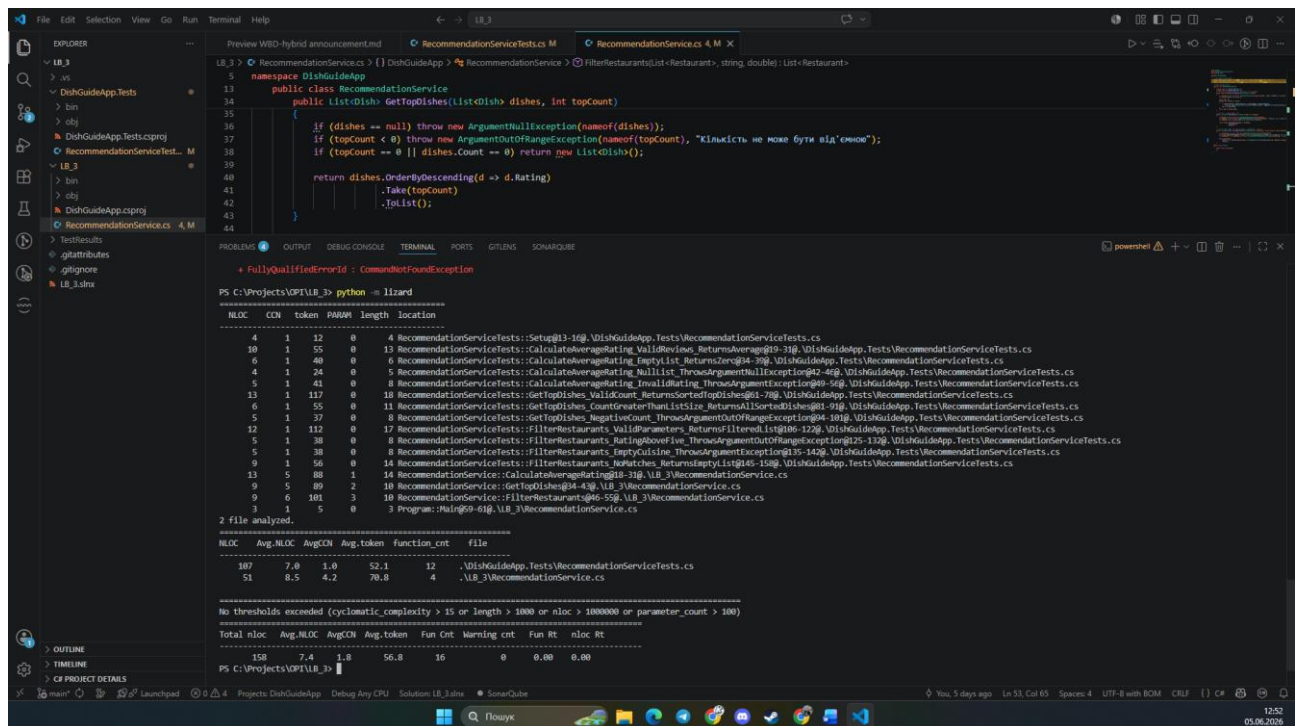


6. Порівняння метрик «ДО / ПІСЛЯ»: кількість SonarLint issues, Ruff/ESLint warnings, цикломатична складність.

До:



Після:



7. Підсумкова рефлексія: 3–5 речень про найцінніший досвід, отриманий під час Code Review.

Під час виконання цієї лабораторної роботи найціннішим досвідом для мене стала можливість практично застосувати інструменти статичного аналізу та метрик для об'єктивної оцінки якості коду. Проводячи Code Review чужого проекту на GitHub, я усвідомив, наскільки важливо дотримуватися принципів Clean Code, уникаючи «магічних чисел» та правильно обробляючи потенційні винятки. Крім того, рефакторинг власного коду за допомогою декларативних LINQ-запитів із подальшим регресійним тестуванням наочно показав мені, як можна зменшити цикломатичну складність без ризику зламати існуючий функціонал. Цей процес значно підвищив мою впевненість у написанні надійного, чистого та підтримуваного C#-коду.

Гілка з переробленим кодом на Гітхаб:

https://github.com/vladyslavstepanenko25/DishGuide_Lab3/tree/LB_4